

Blame & bisect

A regression shows up in simulation, when did it start, and who touched that line?

Blame the line, bisect the range

- Run blame on a file to see hash, author
- Answer good or bad until Git names the first bad commit
- Reset bisect when you are done so you are not stuck in detached state

Browser lab

Digital Design and Verification Platform

Home / Tools / Blame & bisect

Home

Tools

INTERACTIVE TOOL

Blame & bisect

See who last touched each line, then binary-search history to find the commit that introduced a bug — without a real remote.

Starter example: Known good a100, HEAD bad — bisect to find d400, then read blame for the BUG line.

Reset lab

Challenges 0 / 22 cleared

First bad id: What is the first bad commit id? (bisect until found, or reason from the timeline).

Answer

Show hint

Check

Next

Idle

First bad id

Who introduced the bug?

Bad commit message

Finish bisect

Blame BUG comment

BUG line author

Blame en line

en author

enable when high

docs author

module line

rst line

15:0 widen

Last good

HEAD id

HEAD author

Commit count

Bad commit count

Bob's commits

Alice's commits

Carol's only

Reset range

git blame counter.v (at HEAD)

a100	alice	module counter;
c300	alice	logic [15:0] q;
d400	carol	// BUG: reset clears to 1
b200	bob	logic rst;
e500	bob	logic en;
f600	alice	// enable when high
a100	alice	endmodule

git bisect

Known good: a100. HEAD is bad. Mark the midpoint good/bad until the first bad commit appears.

Bisecting: mid c300 (widen bus) · range a100..f600

a100

b200

c300

d400

e500

f600

good

bad

reset bisect

Blame and bisect lab starter

learn_git — blame bisect

Real Git practice

```
wsl - module14-blame-bisect/examples/blame_bisect

# Track A - real Unix
# real shell session (Track A)
# practice repo: /tmp/git-bb-hPjj5i

$ git init

$ echo "v1" > notes.md

$ git add notes.md

$ git commit -m "init"

$ echo "v2 ok" >> notes.md

$ git add notes.md

$ git commit -m "add v2"

$ echo "BUG" >> notes.md

$ git add notes.md

$ git commit -m "introduce bug"

$ git blame notes.md
^4c28989 (Demo 2026-07-20 14:36:45 +0800 1) v1
dde72670 (Demo 2026-07-20 14:36:45 +0800 2) v2 ok
8d58906d (Demo 2026-07-20 14:36:45 +0800 3) BUG

$ git bisect start

$ git bisect bad

$ git bisect good HEAD~2
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[dde7267070e655309e4c223c13c8ce75974f9868] add v2

$ git bisect run ./test_bug.sh
running './test_bug.sh'
8d58906de60452cb41d2ea97d92c57alc3553d0 is the first bad commit
commit 8d58906de60452cb41d2ea97d92c57alc3553d0
Author: Demo <demo@local>
Date:   Mon Jul 20 14:36:45 2026 +0800

    introduce bug

notes.md | 1 +
1 file changed, 1 insertion(+)
bisect found first bad commit
$ git bisect reset
```

Real shell — blame and bisect run

Real Git practice — try these

```
# git init — create a new repository here  
git init
```

```
# echo ... > notes.md — first commit (known good)  
echo "v1" > notes.md
```

```
git add notes.md
```

```
git commit -m "init"
```

```
# echo ... >> notes.md — still good  
echo "v2 ok" >> notes.md
```

```
git add notes.md
```

```
git commit -m "add v2"
```

Pitfalls to watch

- Blame shows the last touch, not every historical edit, use log or show for deeper history
- Bisect needs a reliable good and bad test; flaky tests give flaky answers
- Always bisect reset when finished
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, finish a bisect to the first bad commit and read blame on the bug line
- On real Git, run blame and a short bisect on your practice tree
- When you are ready